Simulation Study for Pseudo-value Regression

FengQi (Cynthia) Song

2026-03-09

Contents

1 Objective **1**

1.1 Sanity check 1

1.2 Monte Carlo validation 2

1.3 Sensitivity analysis(different evaluation time) 3

1.4 Pseudo-value Regression Model 5

1.5 Type I error 6

1.6 Power 7

---

1 Objective

1.1 Sanity check

```r
library(MCC)
source("01_generate_data.R")
t_eval <- 5
pseudo <- MCC::GenPseudo(data = dat, tau = t_eval)

str(pseudo)
```

```
## 'data.frame':    200 obs. of  3 variables:
##  $ idx   : num  1 2 3 4 5 6 7 8 9 10 ...
##  $ psi   : num  1.128 0.18 3.197 -0.943 1.444 ...
##  $ pseudo: num  2.979 2.031 5.048 0.907 3.295 ...
```

```r
head(pseudo)
```

```
##   idx        psi      pseudo
## 1   1  1.1281052  2.97897972
## 2   2  0.1799977  2.03087223
## 3   3  3.1966839  5.04755839
```

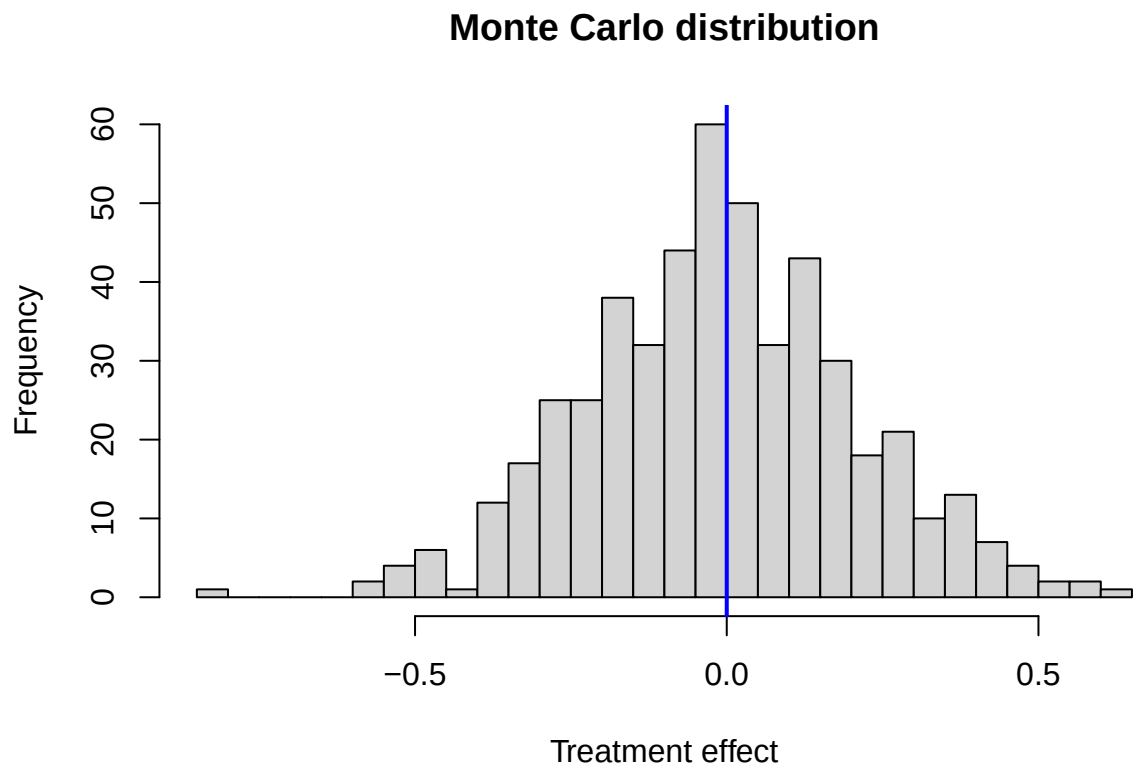
```
## 4   4 -0.9433789  0.90749554
## 5   5  1.4441362  3.29501064
## 6   6 -1.9433789 -0.09250446
```

```r
AX <- unique(dat[, c("idx", "A", "X")])
merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

tapply(merged$pseudo, merged$A, mean)
```

```
##        0        1
## 1.891585 1.810164
```

1.2 Monte Carlo validation

```r
library(MCC)
source("01_generate_data.R")

set.seed(123)

B <- 500
delta_hat <- numeric(B)

for (b in 1:B) {
  source("01_generate_data.R")
  pseudo <- MCC::GenPseudo(data = dat, tau = 5)

  AX <- unique(dat[, c("idx", "A")])
  merged <- merge(AX, pseudo[, c("idx", "pseudo")], by = "idx")

  delta_hat[b] <- mean(merged$pseudo[merged$A == 1]) -
                  mean(merged$pseudo[merged$A == 0])
}

hist(delta_hat, breaks = 30,
     main = "Monte Carlo distribution",
     xlab = "Treatment effect")
abline(v = 0, col = "blue", lwd = 2)
```



This distribution is centered at zero, suggesting no bias

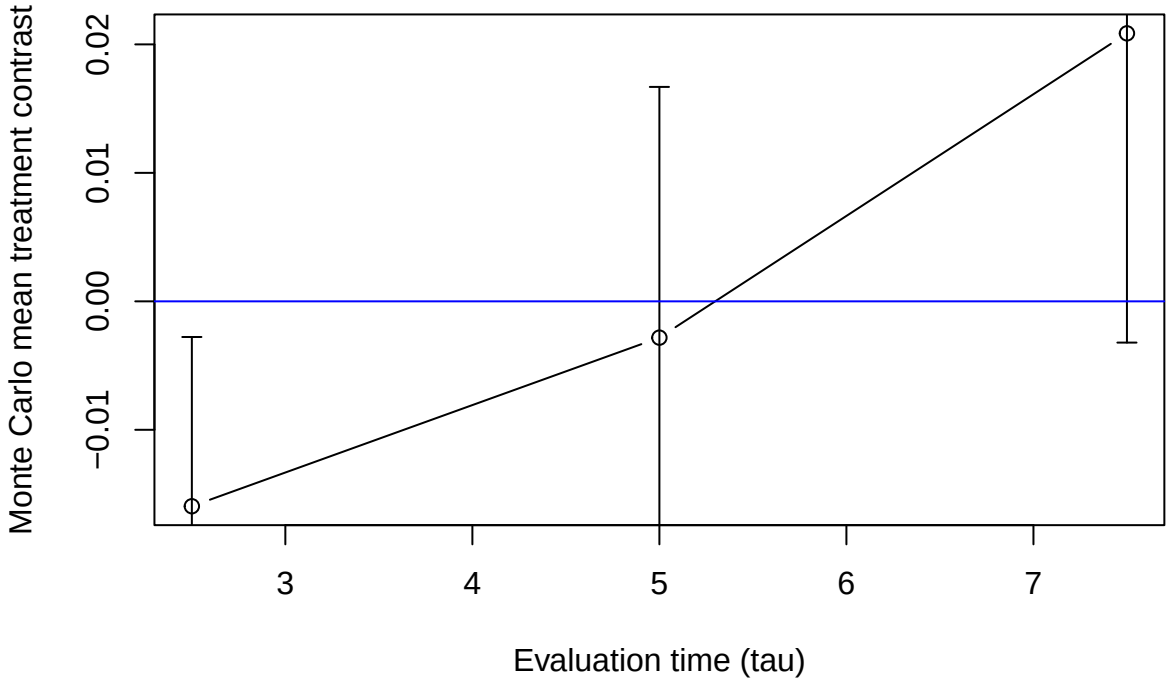
1.3 Sensitivity analysis(different evaluation time)

```
set.seed(123)

B <- 500
t_list <- c(2.5, 5, 7.5)

out_time <- data.frame(
  t_eval = t_list,
  mean_diff = NA_real_,
  sd_diff = NA_real_,
  se_diff = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  delta_tau <- numeric(B)

  for (b in 1:B) {   #Suggested by Generative AI
```

```r
    source("01_generate_data.R")

    pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    merged_j <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

    delta_tau[b] <- mean(merged_j$pseudo[merged_j$A == 1]) -
                    mean(merged_j$pseudo[merged_j$A == 0])
  }

  out_time$mean_diff[j] <- mean(delta_tau)
  out_time$sd_diff[j]   <- sd(delta_tau)
  out_time$se_diff[j]   <- out_time$sd_diff[j] / sqrt(B)
  out_time$lower[j]     <- out_time$mean_diff[j] - 1.96 * out_time$se_diff[j]
  out_time$upper[j]     <- out_time$mean_diff[j] + 1.96 * out_time$se_diff[j]
}

out_time
```

```
##   t_eval    mean_diff   sd_diff     se_diff        lower        upper
## 1    2.5 -0.015950457 0.1502987 0.006721563 -0.029124722 -0.002776193
## 2    5.0 -0.002826793 0.2226700 0.009958107 -0.022344683  0.016691098
## 3    7.5  0.020859660 0.2746303 0.012281842 -0.003212751  0.044932071
```

```r
plot(out_time$t_eval, out_time$mean_diff, type = "b",
     xlab = "Evaluation time (tau)",
     ylab = "Monte Carlo mean treatment contrast")

arrows(
  x0 = out_time$t_eval,
  y0 = out_time$lower,
  x1 = out_time$t_eval,
  y1 = out_time$upper,
  angle = 90,
  code = 3,
  length = 0.05
)

abline(h = 0, col = "blue")
```



1.4 Pseudo-value Regression Model

```
# Single-run regression across tau

t_list <- c(2.5, 5, 7.5)
AX <- unique(dat[, c("idx", "A")])

reg_out <- data.frame(
  t_eval = t_list,
  beta_A = NA,
  p_A = NA
)

for (j in seq_along(t_list)) {
  t_eval <- t_list[j]
  pseudo_j <- MCC::GenPseudo(data = dat, tau = t_eval)
  reg_dat <- merge(AX, pseudo_j[, c("idx", "pseudo")], by = "idx")

  fit <- lm(pseudo ~ A, data = reg_dat)
  s <- summary(fit)$coefficients

  reg_out$beta_A[j] <- s["A", "Estimate"]
  reg_out$p_A[j] <- s["A", "Pr(>|t|)"]
}
```

```r
reg_out
```

```
##   t_eval    beta_A       p_A
## 1    2.5 0.1047129 0.4294708
## 2    5.0 0.2232207 0.2802008
## 3    7.5 0.1875965 0.4837882
```

1.5 Type I error

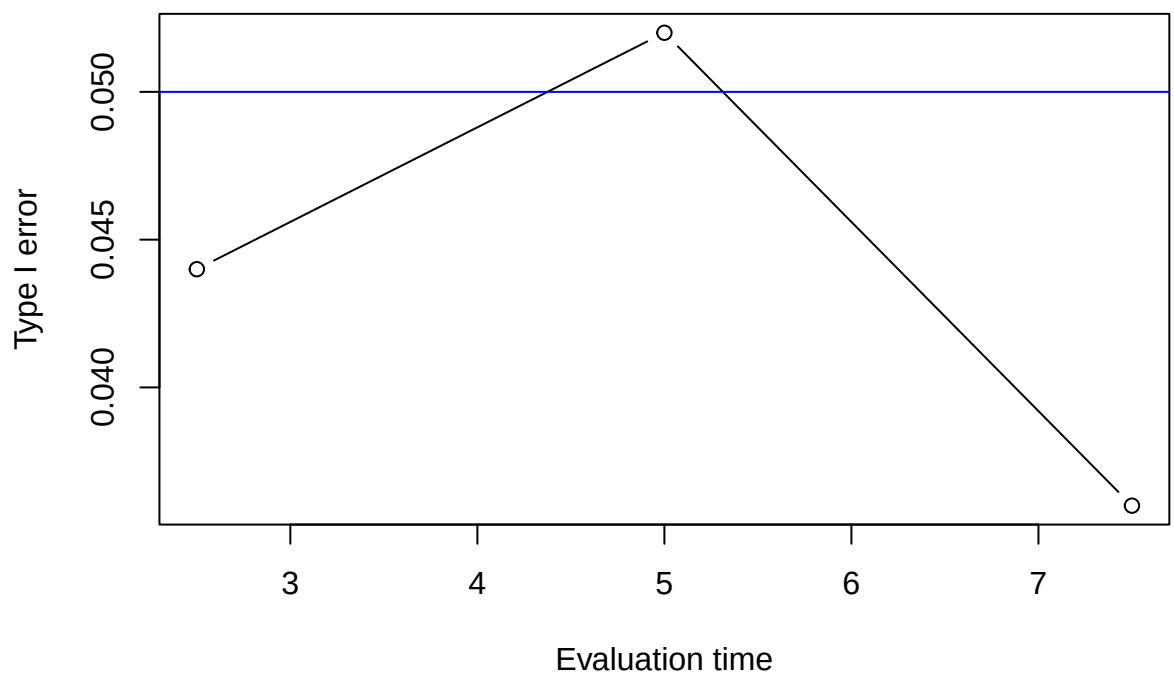
```r
set.seed(123)

B <- 500
alpha <- 0.05
t_list <- c(2.5, 5, 7.5)

type1_result <- data.frame(
  t_eval = t_list,
  reject_rate = NA,
  mean_beta = NA
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {

    alphaA <- 0
    source("01_generate_data.R")

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)

    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }

  type1_result$reject_rate[j] <- mean(reject)
  type1_result$mean_beta[j] <- mean(beta)
}
```

```r
type1_result
```

```
##   t_eval reject_rate    mean_beta
```

```
## 1    2.5       0.044 -0.015950457
## 2    5.0       0.052 -0.002826793
## 3    7.5       0.036  0.020859660
```

```r
plot(type1_result$t_eval, type1_result$reject_rate, type = "b",
     xlab = "Evaluation time",
     ylab = "Type I error")
abline(h = 0.05, col = "blue")
```



1.6 Power

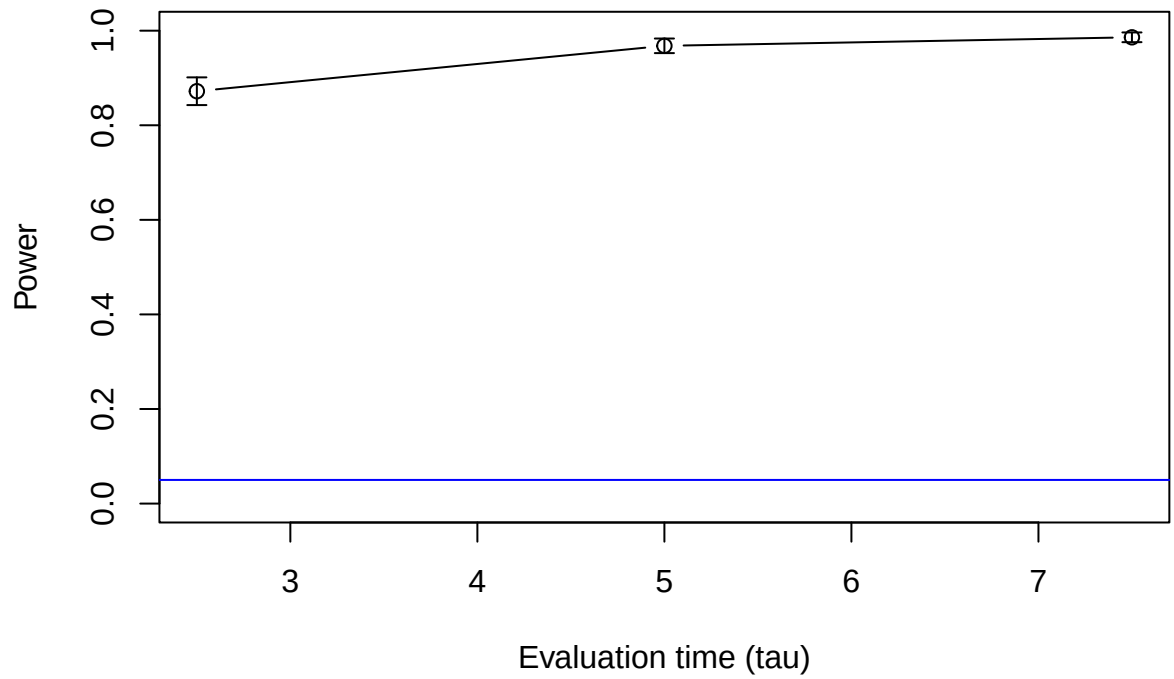
```r
# Type II error
set.seed(123)

B <- 500
alpha <- 0.05
t_list <- c(2.5, 5, 7.5)

alphaA <- log(1.5)

power_out <- data.frame(
  t_eval = t_list,
  power = NA_real_,
```

```r
  mean_beta = NA_real_,
  se_power = NA_real_,
  lower = NA_real_,
  upper = NA_real_
)

for (j in seq_along(t_list)) {

  t_eval <- t_list[j]
  reject <- logical(B)
  beta <- numeric(B)

  for (b in 1:B) {

    source("01_generate_data.R")
    if (b == 1 && j == 1) {
      cat("alphaA in global env =", alphaA, "\n")
      cat("beta_event used =", paste(beta_event, collapse=", "), "\n")
    }

    pseudo_b <- MCC::GenPseudo(data = dat, tau = t_eval)
    AX <- unique(dat[, c("idx", "A")])
    reg_dat <- merge(AX, pseudo_b[, c("idx", "pseudo")], by = "idx")

    fit <- lm(pseudo ~ A, data = reg_dat)
    s <- summary(fit)$coefficients

    beta[b] <- s["A", "Estimate"]
    reject[b] <- s["A", "Pr(>|t|)"] < alpha
  }

  power_out$power[j] <- mean(reject)
  power_out$mean_beta[j] <- mean(beta)

  power_out$se_power[j] <- sqrt(power_out$power[j] * (1 - power_out$power[j]) / B)
  power_out$lower[j] <- max(0, power_out$power[j] - 1.96 * power_out$se_power[j])
  power_out$upper[j] <- min(1, power_out$power[j] + 1.96 * power_out$se_power[j])
}
```

```
## alphaA in global env = 0.4054651
## beta_event used = 0.405465108108164, 0.22314355131421
```

```r
power_out
```

```
##   t_eval power mean_beta    se_power     lower     upper
## 1    2.5 0.872 0.5028479 0.014940950 0.8427157 0.9012843
## 2    5.0 0.968 0.9596981 0.007870959 0.9525729 0.9834271
## 3    7.5 0.986 1.4237185 0.005254332 0.9757015 0.9962985
```

```r
plot(power_out$t_eval, power_out$power, type = "b",
     ylim = c(0, 1),
     xlab = "Evaluation time (tau)",
```

```r
     ylab = "Power")

arrows(
  x0 = power_out$t_eval,
  y0 = power_out$lower,
  x1 = power_out$t_eval,
  y1 = power_out$upper,
  angle = 90,
  code = 3,
  length = 0.05
)

abline(h = alpha, col = "blue")
```